Smart Workspace Manager

Complete Python Learning Project - Updated Plan with SQL, Auth, CI/CD, and React DOM Basics

Version	3.0
Updated	26 July 2026
Environment	Python 3.12, venv, pip
Cloud strategy	Azure free-first for personal use and testing

Phase 1: Streamlit learning product with explicit raw SQL + SQLAlchemy practice.
Phase 2: FastAPI + React upgrade with Azure SQL, auth, authorization, and GitHub Actions.
Free-first Azure deployment for personal use and testing.

Prepared as an independent Python learning project.

Contents

1. Executive Summary

Smart Workspace Manager is one large, independent Python learning project that grows from a Python-first web application into a modern full-stack system. It is designed for a learner who already understands programming concepts through Java and needs practical exposure to Python syntax, Pythonic design, automation, data analysis, SQL, databases, APIs, testing, frontend-backend integration, CI/CD, and cloud deployment.

The project keeps one stable repository containing one backend/ folder and one frontend/ folder. Phase 1 uses Streamlit as the active UI while the reusable Python business logic remains in backend/services/. Phase 2 adds FastAPI, React/Vite, Azure SQL Database, authentication, ownership-based authorization, and GitHub Actions without rewriting the service layer.

Final direction: Use Python 3.12 with venv and pip. Do not use Conda. Docker is optional after the non-containerized deployment works. Use SQLAlchemy professionally, but learn raw SQL first for each important database feature.

2. Final Decisions and Revision Summary

Decision area	Selected decision	Reason
Project structure	One backend/ and one frontend/ from the beginning	Prevents phase duplication and supports direct reuse.
Phase 1 UI	Streamlit	Fast Python-first UI for completing an MVP in 14 days.
Phase 2 UI	React with Vite	Modern frontend that calls FastAPI through HTTP/JSON.
Phase 2 backend	FastAPI	Reuses Phase 1 services and introduces APIs, validation, async concepts, and protected routes.
Database	SQLite locally first; Azure SQL Database for deployed Phase 2	Keeps local setup simple while giving a managed cloud database for testing.
SQL learning method	Raw SQL first, SQLAlchemy second	Ensures the ORM does not block learning SQL.
Authentication	Personal user login with hashed password and token/session	Covers secure login basics.
Authorization	Ownership-based access control	Users can access only their own files, analyses, and reports.
Azure database	Create and connect Azure SQL Database in Phase 2	Covers cloud DB creation, connection strings, migrations, and smoke testing.
CI/CD	Basic GitHub Actions workflows	Runs backend tests, builds frontend, and deploys to Azure.
DOM/browser basics	React events, forms, controlled inputs, conditional rendering, and limited useRef	Covers practical browser interaction without deep vanilla JavaScript.
Environment	Python 3.12 + venv + pip	Simple, standard, and sufficient for the project.
Conda	Not used	Adds no required benefit for the selected libraries and learning goals.
Docker	Optional later	Avoids deployment complexity during the 21-day plan.
Cloud	Azure free-first	Appropriate for one-person testing; paid services are introduced only before publication.

3. Project Vision, Objectives, and Scope

3.1 Vision

Build one complete web-based workspace application that teaches Python progressively while producing a useful system for uploading, organizing, analyzing, and reporting on files and datasets.

3.2 Objectives

- Learn Python syntax through real application code rather than isolated exercises.
- Practice modular programming, object-oriented design, exceptions, type hints, files, and configuration.
- Build automation for file validation, categorization, organization, cleanup, and logging.
- Use pandas, NumPy, Plotly, or matplotlib in a controlled data analysis workflow.
- Learn raw SQL querying before implementing the same database operations with SQLAlchemy.

- Use SQLite locally, then connect the deployed Phase 2 system to Azure SQL Database.
- Expose reusable Python services through FastAPI endpoints.
- Build a React frontend that consumes the API.
- Practice authentication using password hashing and token/session-based protected routes.
- Practice basic authorization using ownership checks for files, reports, and analyses.
- Create Azure SQL Database through Azure and connect it to FastAPI.
- Implement basic CI/CD using GitHub Actions for backend tests, backend deployment, frontend build, and frontend deployment.
- Practice basic DOM/browser interaction through React events, controlled inputs, conditional rendering, and limited direct element access using useRef.
- Test service, database, API, authentication, authorization, and deployment behavior with pytest and cloud smoke tests.

3.3 In Scope

- Dashboard, file upload, file library, automatic file organization, CSV analysis, visualizations, reports, settings, logs.
- Streamlit UI in Phase 1 and React UI in Phase 2.
- FastAPI REST endpoints, Pydantic validation, SQLAlchemy, Alembic, authentication, authorization, GitHub Actions, and cloud deployment.
- Small personal test datasets for analysis and reporting.

3.4 Out of Scope

- A production multi-tenant SaaS platform during the learning timeline.
- Large-scale data processing or distributed computing.
- Mobile or desktop native applications.
- Payment processing, enterprise role hierarchies, Kubernetes, or complex microservices.
- Advanced vanilla DOM programming beyond basic React/browser concepts.
- Connections to any previous academic, research, organizational, or personal project.

4. Two-Phase Development Strategy

Phase	Duration	Active UI	Backend style	Database	Primary result
Phase 1	14 days	Streamlit	Modular Python services	SQLite	Working learning MVP deployed to Azure App Service F1.
Phase 2	6 build days + 1 buffer day	React/Vite	FastAPI REST API	Azure SQL Database	Professional separated frontend/backend system with auth, authorization, and basic CI/CD on free Azure services.

Phase 1 must be complete enough to demonstrate independently. Phase 2 is an upgrade, not a restart. The service layer, utility functions, database models, analysis logic, report logic, SQL knowledge, and tests remain reusable.

5. System Features and End-to-End Flow

Module	Purpose
Dashboard	Summary cards, recent uploads, analysis counts, report counts, storage use, and quick actions.
File Upload	Validate file extension and size, save safely, calculate metadata, and create a database record.
File Organizer	Classify files by type or date, rename safely, move files, and create automation logs.
File Library	Search, filter, inspect, download, rename, and delete records and files.
CSV Analyzer	Preview data, inspect data types, detect missing values and duplicates, calculate statistics, and select columns.
Data Cleaning	Apply simple cleaning actions and export a cleaned dataset without overwriting the original.
Visualization	Create bar, line, histogram, scatter, and missing-value charts where appropriate.
Report Generator	Create CSV, Excel, and text/HTML summaries; PDF remains optional.
Authentication	Personal user login, password hashing, token/session creation, and protected API routes.

Module	Purpose
Authorization	Ownership checks so each user can access only their own files, analyses, and reports.
CI/CD	GitHub Actions runs backend tests, builds React, and deploys backend/frontend to Azure.
Settings and Logs	Manage paths, safe limits, processing logs, and deployment settings.

5.2 End-to-End User Flow

1. The user opens the web application.
2. In Phase 2, the user logs in and receives a token/session.
3. The dashboard loads system summaries from the database.
4. The user uploads a file.
5. The file is validated, stored safely, and recorded in the database.
6. Ownership is saved so the file is linked to the logged-in user.
7. The organizer classifies the file and places it in the correct managed location.
8. For a CSV file, the analyzer returns schema information, missing values, duplicates, statistics, and chart-ready results.
9. The user applies optional cleaning actions and exports a cleaned copy.
10. The report service creates a downloadable summary.
11. In Phase 2, React requests all major operations through protected FastAPI endpoints.
12. GitHub Actions automates the basic test/build/deploy workflow.

6. Architecture and Design Principles

6.1 Phase 1 Runtime Flow

Streamlit frontend
frontend/streamlit_app.py and frontend/pages/ collect input and display results.

|

Service layer
backend/services/ contains file, analysis, report, automation, and storage logic.

|

Data access layer
backend/database/ performs SQLite CRUD through repositories.

|

SQL learning layer
Important raw SQL examples are written and understood before the SQLAlchemy equivalent.

|

Storage
Local data/ folders during development; /home/data in Azure App Service for small personal test files.

6.2 Phase 2 Runtime Flow

React frontend
Hosted on Azure Static Web Apps Free and sends HTTPS/JSON requests.

|

FastAPI API layer
backend/main.py and backend/routes/ validate requests and call existing services.

|

Auth and authorization layer
Dependencies verify token/session and enforce ownership checks.

|

Service layer
The same backend/services/ modules from Phase 1 remain the business logic.

|

Database layer
SQLAlchemy connects to SQLite locally and Azure SQL Database in the deployed system.

|

CI/CD layer
GitHub Actions runs tests/builds and deploys to Azure.

|

Storage
Small test files remain in App Service persistent storage; Blob Storage is introduced before publication.

6.3 Mandatory Design Rules

- **Thin UI:** Streamlit and React files must not contain core file, analysis, report, or database logic.
- **Reusable services:** Each service accepts normal Python values or file paths and returns structured results.
- **SQL-first learning:** For important database features, understand the raw SQL query before implementing it with SQLAlchemy.
- **Configuration-driven:** Paths, file limits, database URLs, secrets, and cloud settings come from environment variables or settings.
- **Preserve originals:** Cleaning and organization operations must not silently overwrite original uploads.
- **Auth separation:** Authentication and authorization logic live in backend services/dependencies, not React pages.
- **CI/CD safety:** Secrets are stored in GitHub Secrets or Azure app settings, never in code.
- **One active cloud backend:** The Phase 1 App Service is repurposed for FastAPI in Phase 2 rather than keeping two F1 backend apps running.

7. Phase 1 Specification

Phase 1 is a 14-day MVP built primarily in Python. Streamlit acts only as the presentation layer. The service and database code is written with Phase 2 reuse in mind, and important database operations include raw SQL practice before SQLAlchemy implementation.

7.1 Phase 1 Technology Stack

Area	Selection
Language	Python 3.12
Environment	venv + pip
UI	Streamlit
Database	SQLite with SQLAlchemy and raw SQL learning examples
Data analysis	pandas and NumPy
Charts	Plotly; matplotlib optional
Excel	openpyxl
Configuration	python-dotenv or pydantic-settings later
Testing	pytest and pytest-cov
Cloud	Azure App Service F1

Area	Selection
Cloud file location	Persistent /home/data folders for small test files

7.2 Phase 1 Functional Scope

Feature	Phase 1 requirement
Dashboard	Counts, recent records, quick links, status cards, and raw SQL count/group-by examples.
Upload	CSV, JSON, TXT, PDF, images, and common documents with size/type validation.
Organizer	Classification, safe names, category/date folders, logs, and duplicate-name handling.
Metadata	File name, stored name, extension, size, path, status, and timestamps.
SQL practice	CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, WHERE, ORDER BY, COUNT, and GROUP BY examples.
Analyzer	Preview, data types, missing values, duplicates, summary statistics, and column selection.
Cleaning	Drop duplicates, selected missing-value handling, basic type conversion, and export.
Charts	Basic interactive charts chosen according to column types.
Reports	CSV/Excel/text or HTML summary export.
Testing	Unit tests for services plus database integration tests.
Deployment	Streamlit on Azure App Service F1 with a custom startup command.

7.3 Phase 1 Completion Criteria

- The application starts locally through Streamlit using the project venv.
- File upload, metadata storage, organization, listing, download, and delete workflows function.
- Raw SQL examples and equivalent SQLAlchemy repository functions exist for the main file/database operations.
- CSV analysis and at least three chart types work on small test datasets.
- Cleaned data and an analysis report can be downloaded.
- Tests pass for the main service functions.
- Core logic is not embedded inside Streamlit button blocks.
- The app is deployed to Azure App Service F1 and works within free-tier limits.

8. Phase 2 Specification

Phase 2 converts the system into a separated web application using FastAPI + React. Django is excluded from the active plan to keep the seven-day upgrade achievable.

8.1 Phase 2 Additions

Addition	Purpose
Raw SQL practice	Write and understand SQL queries before implementing equivalent SQLAlchemy queries.
FastAPI application	Application entry point, route registration, CORS, error handlers, and health endpoint.
API routes	Files, analyses, reports, authentication, authorization-aware access, and settings endpoints.
Pydantic schemas	Request and response models with type validation.
React/Vite frontend	Pages, components, routing, forms, API client, chart rendering, and error states.
DOM/browser basics	Form events, button events, controlled inputs, conditional rendering, and one simple useRef use case.
Azure SQL Database	Managed relational database for the deployed system under the available free offer, if supported by the subscription.
Alembic	Database schema migrations.
Authentication	Personal account login, password hashing, token/session creation, and protected routes.
Authorization	Ownership checks for files, reports, and analysis jobs.

Addition	Purpose
GitHub Actions CI/CD	Backend tests and deployment; frontend build and deployment; secrets stored safely.
API tests	FastAPI tests for success, validation, authentication, authorization, and errors.
Cloud split	React on Static Web Apps Free; FastAPI on the repurposed App Service F1.

8.2 Phase 2 API Outline

Method	Endpoint	Purpose
GET	/health	Backend status check.
POST	/api/auth/login	Authenticate the personal user.
GET	/api/auth/me	Return the current authenticated user.
POST	/api/files	Upload a file and create metadata owned by the user.
GET	/api/files	List and filter files owned by the user.
GET	/api/files/{id}	Read one owned file record.
DELETE	/api/files/{id}	Delete an owned file and metadata safely.
POST	/api/analyses	Analyze a selected owned CSV.
POST	/api/analyses/{id}/clean	Create a cleaned copy if the user owns the source.
POST	/api/reports	Generate a report for an owned file/analysis.
GET	/api/reports	List generated reports owned by the user.
GET	/api/dashboard	Return dashboard summary data for the user.

8.3 Phase 2 Completion Criteria

- React is the active UI and all major functions are requested through FastAPI.
- FastAPI reuses Phase 1 services instead of duplicating business logic.
- Raw SQL examples and SQLAlchemy equivalents are documented.
- The deployed backend uses Azure SQL Database or a clearly documented fallback if the free offer is unavailable for the subscription.
- The React frontend is hosted on Azure Static Web Apps Free.
- The FastAPI backend is hosted on Azure App Service F1.
- Login and protected routes work for the personal testing account.
- Ownership-based authorization prevents access to another user's records.
- Basic GitHub Actions backend and frontend workflows run successfully.
- React UI demonstrates forms, events, controlled inputs, conditional rendering, and a simple useRef example.
- API tests and service tests pass.
- The legacy Streamlit app remains available in the repository for reference.

9. Azure Free-First Deployment Strategy

Purpose of the free architecture: The system is for one-person learning and testing. It intentionally accepts sleep, quota, storage, performance, free-offer eligibility, and no-SLA limitations.

9.1 Phase 1 Cloud Architecture

Browser
The user accesses the Streamlit URL.

|

Azure App Service F1
Runs the Streamlit application and installs Python packages from requirements.txt.

|

SQLite + /home/data
Stores a small local database and small uploaded/processed test files in persistent App Service storage.

|

GitHub
Stores source code and supports manual or workflow-based deployment.

9.2 Phase 2 Cloud Architecture

Azure Static Web Apps Free
Hosts the React/Vite build for a personal project.

|

Azure App Service F1
Runs FastAPI. The existing Phase 1 App Service is repurposed for the API.

|

Azure SQL Database free offer
Stores users, file metadata, analyses, reports, and logs when available for the subscription.

|

GitHub Actions
Runs tests/builds and deploys to Azure using repository secrets.

|

App Service /home/data
Stores only small personal test files until the project is prepared for publication.

9.3 Free-Tier Constraints and Eligibility Notes

Service	Relevant limitation or allowance
App Service F1	Shared compute with strict CPU/storage limits and no production SLA. Keep datasets and processing workloads small.
Azure Static Web Apps Free	Suitable for personal/static frontend hosting and GitHub-based deployment workflows.
Azure SQL Database free offer	Provides a monthly free allowance where available; verify eligibility in the actual Azure subscription before relying on it.
Student subscription note	Some Azure student/student starter offers may have compatibility or regional constraints. Confirm availability in the portal.
Operational implication	Keep datasets, reports, and concurrent work small; delete unnecessary files and monitor quotas.

9.4 Cost-Control Rules

- Create all Azure resources inside one resource group such as rg-smart-workspace-manager.
- Select F1 explicitly for App Service and Free explicitly for Static Web Apps.
- Select the Azure SQL free-offer configuration only if it is available for your subscription.
- Set the free-limit behavior to stop or pause rather than continue into billable use where that option is available.
- Create budget alerts even when the intended cost is zero.

- Do not create Azure Container Registry, Kubernetes, premium databases, or paid monitoring during the learning plan.
- Delete unused resources and review Cost Analysis after each deployment exercise.
- Treat all free limits as subscription- and region-dependent until confirmed in the Azure portal.

10. Database and Storage Roadmap

10.1 Database Progression

Environment	Database	Reason
Local Phase 1	SQLite	Fast setup and simple debugging.
Azure Phase 1	SQLite file under /home/data	Acceptable only for one-person MVP testing.
Local Phase 2	SQLite through SQLAlchemy	Keeps local setup easy while the ORM matches the cloud architecture.
Azure Phase 2	Azure SQL Database	Managed cloud database for users, metadata, analyses, reports, and logs when the free offer is available.
Fallback if free SQL unavailable	SQLite for private cloud demo or defer Azure SQL until paid/eligible	Document the limitation clearly rather than guessing.
Future publication	Azure SQL paid usage or another selected managed tier	Higher capacity, reliability, backups, and operational controls.

10.2 Core Entities

Entity	Purpose
users	Personal login in Phase 2.
files	Original/stored names, type, size, path or storage key, status, timestamps, and owner/user_id.
analysis_jobs	File reference, owner, status, requested options, summary, timestamps, and errors.
reports	Analysis/file reference, owner, report type, path or key, status, and creation time.
automation_logs	Action, target, status, message, and timestamp.
settings	Safe configurable values.

10.3 Storage Abstraction

All file access should pass through backend/services/storage_service.py. This service hides whether files are stored in the local project, App Service /home/data, or Azure Blob Storage. The initial implementation uses local paths. A future AzureBlobStorageProvider can be added without rewriting upload, analysis, and report logic.

```
Local development:
STORAGE_PROVIDER=local
DATA_ROOT=./data

Azure free testing:
STORAGE_PROVIDER=local
DATA_ROOT=/home/data

Future publication:
STORAGE_PROVIDER=azure
AZURE_STORAGE_CONNECTION_STRING=...
```

10.4 SQL Learning Rule

Database learning rule: For every important database feature, first understand the raw SQL query, then implement the same behavior using SQLAlchemy. SQLAlchemy is used professionally, but it must not hide SQL learning.

Feature	Raw SQL concept	SQLAlchemy implementation
File table setup	CREATE TABLE	ORM model definition + migration

Feature	Raw SQL concept	SQLAlchemy implementation
File upload metadata	INSERT	create_file repository function
File list	SELECT	list_files repository function
File search/filter	WHERE, LIKE, ORDER BY	filtered query
File status update	UPDATE	update_file_status function
Delete file record	DELETE	delete_file function
Dashboard counts	COUNT, GROUP BY	aggregate query
File reports	JOIN	relationship query
User-owned files	WHERE user_id = ?	ownership filter in repository/service

11. Technology Stack and Packages

11.1 Phase 1 Python Packages

Package	Purpose
streamlit	Phase 1 web interface.
pandas	Tabular data loading, cleaning, transformation, and analysis.
numpy	Numerical operations and array support.
plotly	Interactive visualizations.
matplotlib	Optional basic plotting and report images.
SQLAlchemy	ORM and database abstraction after raw SQL understanding.
openpyxl	Excel import/export.
python-dotenv	Local .env configuration.
pytest	Testing.
pytest-cov	Coverage reporting.

11.2 Phase 2 Python Additions

Package	Purpose
fastapi	REST API framework.
uvicorn[standard]	Local ASGI server.
gunicorn	Linux process manager for Azure App Service.
pydantic-settings	Typed configuration.
python-multipart	File upload handling.
alembic	Schema migrations.
pyodbc	Azure SQL connectivity through SQLAlchemy.
PyJWT	JWT token creation and validation.
pwdlib[argon2]	Password hashing.
httpx	API client and API tests.
pytest-asyncio	Async test support.

11.3 React and CI/CD Tools

Tool/package	Purpose

Tool/package	Purpose
react and react-dom	Frontend runtime.
vite	Development server and production build.
react-router-dom	Client-side routing.
axios	API calls.
react-plotly.js or another chart wrapper	Frontend charts.
vitest and React Testing Library	Optional frontend tests.
GitHub Actions	Backend tests, frontend build, and Azure deployment automation.
GitHub Secrets	Stores Azure publish profile, Static Web Apps token, API URL, and sensitive deployment values.

11.4 Environment Decision

Use venv: Create the environment with py -3.12 -m venv smart-workspace. Conda is unnecessary. Docker is excluded from the required work plan and may be added after the non-containerized Azure deployment works.

12. Security, Testing, and Quality Plan

12.1 Security Basics

- Validate extensions, MIME information where possible, file sizes, and generated file names.
- Never trust a user-provided path; create server-controlled storage paths.
- Keep .env, database credentials, JWT secrets, and connection strings out of Git.
- Hash passwords; never store plain-text passwords.
- Protect private routes using token/session verification.
- Implement authorization checks so users can access only records they own.
- Use 401 Unauthorized for missing/invalid authentication and 403 Forbidden for authenticated access to someone else's resource.
- Restrict CORS to the deployed frontend URL in Phase 2.
- Use parameterized ORM queries/schema validation and avoid unsafe string-built SQL.
- Limit uploaded file size and reject unsupported formats.
- Do not expose internal filesystem paths in API responses.
- Store deployment secrets in Azure app settings and GitHub Secrets, not source code.

12.2 Test Levels

Test level	Scope
Unit tests	Validators, safe naming, category detection, cleaning functions, summary calculations, and report helpers.
SQL learning tests	Validate raw SQL examples and equivalent SQLAlchemy behavior where practical.
Database integration	CRUD, relationships, ownership filters, transaction behavior, and migration checks.
Auth tests	Login success, wrong password, missing token, invalid token, and current-user retrieval.
Authorization tests	User cannot access another user's files, reports, or analyses.
Service integration	Upload + metadata + organization; analysis + report generation.
API tests	Success, validation failure, not found, unauthorized, forbidden, and server error behavior.
Manual UI tests	Navigation, forms, upload states, chart rendering, downloads, auth states, and DOM-related behavior.
CI/CD tests	GitHub Actions runs backend tests and frontend build before deployment.
Cloud smoke tests	Health endpoint, login, one upload, one analysis, one report, one authorization check after deployment.

12.3 Quality Rules

- Use clear names, type hints, docstrings for public functions, and small focused modules.
- Use logging instead of scattered print statements.
- Commit after meaningful milestones and tag Phase 1 and Phase 2 releases.
- Keep requirements explicit and update README run/deploy commands.
- Preserve sample data separately from user-uploaded test data.
- Perform a final clean setup test from a fresh venv.
- Document raw SQL examples, SQLAlchemy equivalents, auth behavior, Azure SQL creation steps, and GitHub Actions workflows.

13. Python Concepts and Technologies Covered

Area	Coverage
Core syntax	Variables, types, strings, numbers, booleans, None, conditions, loops, functions, imports, and modules.
Collections	Lists, dictionaries, tuples, sets, comprehensions, sorting, filtering, and iteration.
Functions	Parameters, defaults, keyword arguments, return values, scope, lambdas where appropriate, and callbacks.
OOP	Classes, objects, constructors, instance methods, composition, inheritance only where justified, and dataclasses.
Errors	Built-in exceptions, custom exceptions, try/except/finally, validation, and error propagation.
Files	pathlib, reading/writing, binary files, CSV, JSON, Excel, safe renaming, moving, and deletion.
Advanced Python	Type hints, decorators through routes, context managers, generators/streaming where useful, and async/await in FastAPI.
Automation	Batch organization, logs, cleanup rules, scripts, and scheduled concepts.
Data science	DataFrames, missing values, duplicates, summaries, transformations, charts, and simple preprocessing.
Databases	Raw SQL querying, CRUD, joins, grouping, relationships, SQLAlchemy ORM, migrations, Azure SQL, and database URLs.
Web development	Streamlit state, REST APIs, FastAPI routes, HTTP status codes, JSON, validation, CORS, authentication, authorization, and frontend integration.
Frontend basics	React components, props, state, events, controlled inputs, conditional rendering, API calls, auth-aware UI, dynamic rendering, and basic DOM access through useRef.
DevOps	Git, GitHub, GitHub Actions, automated backend tests, frontend build, backend deployment, frontend deployment, GitHub Secrets, and Azure environment variables.
Testing	Fixtures, assertions, mocking where needed, API tests, auth tests, authorization tests, CI checks, coverage, and regression prevention.
Cloud	Azure deployment, startup commands, environment variables, quotas, managed database, static hosting, and cost-control checks.

14. Limitations and Topics Outside Scope

Area	Reason
Desktop GUI	Tkinter, PyQt, and Kivy require a separate project.
Mobile development	Professional mobile development is outside the selected stack.
Game development	Pygame and real-time game loops are not represented.
Big data	Spark, Kafka, Airflow, and distributed processing exceed the project scope.
Advanced cloud	Kubernetes, Terraform, service mesh, and multi-region architecture are not needed.
Heavy background processing	Celery/Redis can be introduced later but are not suitable for the free-first timeline.
Enterprise security	The project covers essential app security, not penetration testing or compliance.
Advanced React	React is learned enough to operate this frontend, not as a complete frontend-specialist curriculum.
Advanced DOM manipulation	Basic DOM/browser concepts are covered through React; deep vanilla DOM APIs are outside scope.

Area	Reason
Advanced CI/CD	Basic GitHub Actions are covered; release approvals, staging slots, IaC, and complex pipelines are outside scope.
Python internals	CPython implementation, C extensions, the GIL in depth, and memory internals remain separate topics.

15. Deliverables and Completion Criteria

ID	Deliverable
D1	Git repository with stable combined structure.
D2	Working Streamlit Phase 1 application.
D3	Reusable backend service and database layers.
D4	File management, analysis, and reporting features.
D5	pytest suite and test report.
D6	Azure App Service F1 Phase 1 deployment.
D7	FastAPI backend and documented endpoints.
D8	React/Vite frontend.
D9	Azure SQL database creation, connection, and migrations, or documented eligibility fallback.
D10	Static Web Apps Free frontend and App Service F1 API deployment.
D11	README, project document, folder document, work plan, API notes, screenshots, and known limitations.
D12	Raw SQL query examples and SQLAlchemy equivalents documented.
D13	Authentication and ownership-based authorization implemented.
D14	Basic GitHub Actions backend and frontend workflows created and tested.
D15	React UI demonstrates basic DOM/browser concepts through forms, events, state, conditional rendering, and useRef.
D16	Azure SQL creation/deployment notes, GitHub Secrets list, and CI/CD workflow explanations documented.

16. Risks, Constraints, and Mitigations

Risk	Impact	Mitigation
Scope too large	Features may not finish in 21 days.	Use strict MVP criteria; treat PDF, jobs, and Docker as optional.
Logic mixed into UI	Phase 2 becomes a rewrite.	Require all important operations to live in backend/services/.
ORM hides SQL	SQL learning becomes weak.	For every important query, write raw SQL first and then SQLAlchemy.
Free App Service quota	App stops after quota exhaustion.	Use small datasets and repurpose rather than duplicate F1 apps.
Cloud storage limit	Uploads fill the 1 GB quota.	Set file-size limits, clean test files, and move to Blob Storage only before publication.
Azure SQL eligibility/configuration	Unexpected billing, unavailable free offer, or connectivity issues.	Confirm in Azure portal, choose free offer if available, stop at free limits, and document fallback.
Authentication complexity	Delays final integration.	Implement one personal user and basic protected routes; defer roles, password reset, and email verification.
Authorization bugs	Data isolation may fail.	Enforce owner filters in service/repository layer and test forbidden access.
CI/CD secrets risk	Deployment credentials could leak.	Use GitHub Secrets and Azure app settings only; never commit publish profiles or tokens.
React learning overhead	Phase 2 exceeds seven days.	Use a simple layout and implement only existing Phase 1 features.
DOM scope creep	Frontend becomes a JavaScript side project.	Limit DOM learning to React events, controlled inputs, conditional rendering, and one useRef example.
Package/version conflict	Environment becomes unstable.	Use Python 3.12, venv, pinned dependencies after setup, and a clean install test.

17. Future Publication Upgrade Path

Area	Free testing state	Publication upgrade
Compute	App Service F1	Paid Basic/Standard App Service plan with Always On and higher resources.
Frontend	Static Web Apps Free	Keep Free if sufficient or move to Standard for production requirements.
Database	Azure SQL free allowance or documented fallback	Paid Azure SQL capacity, backups, monitoring, and performance tuning.
File storage	App Service /home/data	Azure Blob Storage with private containers and lifecycle rules.
Secrets	App settings and GitHub Secrets	Azure Key Vault and managed identity.
Monitoring	Basic logs	Application Insights, alerts, dashboards, and retention.
Jobs	Synchronous processing	Azure Functions or a worker/queue system for long jobs.
Deployment	Basic GitHub Actions	Controlled CI/CD, staging, release approvals, and optional Docker.

18. References

- Azure App Service for Linux pricing and F1 limits: https://azure.microsoft.com/en-us/pricing/details/app-service/linux/
- Azure SQL Database free offer: https://learn.microsoft.com/en-us/azure/azure-sql/database/free-offer?view=azuresql
- Azure SQL Database free-offer FAQ: https://learn.microsoft.com/en-us/azure/azure-sql/database/free-offer-faq?view=azuresql
- Azure Static Web Apps hosting plans: https://learn.microsoft.com/en-us/azure/static-web-apps/plans
- Azure Static Web Apps build configuration and deployment token guidance: https://learn.microsoft.com/en-us/azure/static-web-apps/build-configuration
- Deploy Python web apps to Azure App Service: https://learn.microsoft.com/en-us/azure/app-service/quickstart-python
- Configure Python apps on Azure App Service: https://learn.microsoft.com/en-us/azure/app-service/configure-language-python
- GitHub Actions workflows: https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows
- GitHub Actions secrets: https://docs.github.com/en/actions/reference/security/secrets
- Streamlit documentation: https://docs.streamlit.io/
- FastAPI documentation: https://fastapi.tiangolo.com/
- SQLAlchemy documentation: https://docs.sqlalchemy.org/
- pandas documentation: https://pandas.pydata.org/docs/
- pytest documentation: https://docs.pytest.org/